

02-lab-work-speaking-script

S3 Lab Work: Speaking Script

For the one-to-one walkthrough with Prof. Naik. Companion to [the submission document](#).

Covers	S3 Lab work, 40 marks
Format	One-to-one, screen shared
Target length	12 minutes spoken, leaving room for questions

How to use this

The script is written to be **said, not read aloud**. Learn the shape, keep the page open for the numbers. Bold text is the sentence that carries the mark; if you are cut short, say the bold ones.

The rule for the whole meeting: every claim is followed by the thing that proves it, on screen. Never say "we have good code quality." Say what is enforced and let the build be the evidence.

Have open before you start, in this tab order:

1. The repository on GitHub, with the CI checks visible and green
2. `docs/non-functional-requirements.md`
3. `apps/api/src/revix_api/main.py`
4. `pyproject.toml`
5. The live site, already loaded and warm
6. A terminal in the repo root

0. Opening (30 seconds)

Revix is an evidence-traceable review system for Indian cars and two-wheelers. The problem is that a star average hides who rated, what they cared about, and whether they agreed. **We rebuild the verdict from the reviews underneath it and keep every number traceable to the sentences it came from.**

It is running on real data: **15,853 review units across three sources, 143 variants, 42 models**, refreshed by a pipeline that runs itself every night. Today I will go through the four things this submission is marked on: codebase, frameworks, code quality, and the non-functional requirements.

Do not demo the product here. That is the 16 October submission. If he starts clicking, let him, but steer back: "I can show the full flow properly in the demo session; today I would rather show you what is underneath it."

1. Codebase (target 2.5 minutes)

Show: `pyproject.toml`, then the folder tree.

It is a uv workspace: three Python packages and a Next.js app, about **seventeen thousand lines**. Core holds the models and settings. The pipeline does ingest, extraction, resolution and fusion. The API is the serving layer, and it is deliberately the smallest thing here, under nine hundred lines.

Scroll to the dependency comment in `pyproject.toml`.

The one rule is that dependencies run in one direction. The API and the pipeline both depend on core, and core depends on neither. That is what stops the read path and the write path tangling.

And it is enforced by import linting in CI, not by us remembering. That is why it is three packages instead of one: in one package this would be a naming convention, and naming conventions lose. Here the API physically cannot import the pipeline.

Show the database schemas.

The same idea in Postgres: four schemas separated by lifecycle, not by subject. `raw` is what we fetched, `core` is the catalogue and evidence, `analysis` is scores and evaluation runs, `serving` is finished verdicts. You can always tell which layer a table is in and what may write to it. Schema changes go through Alembic, and CI checks the migrations apply, match the models, **and reverse cleanly**.

If he asks whether it actually runs:

Every night, unattended. The last full run took **fifty-three and a half minutes** inside a two-hour timeout. Nobody starts it and nobody watches it.

2. Frameworks with justification (target 3 minutes)

Show: the `docs/adr/` folder.

Rather than justify these in a slide, we wrote them down as decisions. **Nine architecture decision records**, each with the decision, the alternatives, and the consequence we accepted. I will give you the two worth arguing about.

Open ADR 0002.

We use scheduled GitHub Actions instead of Airflow or Prefect. An orchestrator is a server to run, secure and pay for. Our pipeline is one nightly batch: no fan-out, no backfill, no dynamic DAG. Actions already holds our secrets, already runs on a schedule, already gives us logs and alerting.

Choosing Airflow here would have been choosing a tool because it looks professional rather than because the problem needs it. If we ever needed backfill or fan-out, that ADR is where the decision gets revisited.

Open ADR 0004.

The second one is about machine learning, and it is the one I would want you to push on. We built a distant-supervision aspect classifier: lexicon labels train a TF-IDF into a one-vs-rest logistic regression.

It lost to the lexicon it learned from by 0.29 macro F1, so it ships disabled.

We kept the result and the code, and the number is published on our metrics endpoint. **The alternative was shipping a model because we had built one**, and the whole argument of this project is that a number should be earned.

This is your strongest moment. Do not rush it and do not apologise for it.

The rest, quickly, only if there is time:

FastAPI because it is contract-first: the OpenAPI schema is the source of truth and the frontend's TypeScript client is generated from it, so they cannot drift. SQLAlchemy's typed ORM so there is no raw SQL and injection has no surface. Postgres with pgvector and pg_trgm because one datastore covers relational, vector and fuzzy text; a separate vector database would have been a second service to deploy and keep consistent for a workload this size. Next.js App Router because a verdict should be a real shareable URL.

3. Code quality (target 2.5 minutes)

Show: the green CI checks on a pull request.

Six jobs on every pull request. **mypy strict** across all three Python packages, TypeScript strict, ruff for lint and formatting, **246 tests**, a Playwright browser test, and **axe-core against all ten pages** for accessibility, which fails the build on a violation.

Three of these are less usual and worth naming: CI checks that **migrations reverse cleanly**, that the **pipeline runs end to end**, and that **no secret and no raw scraped payload is ever committed**. The repository is public, so that last one is a hygiene gate rather than a preference.

Point at the "committed API types match the API" step.

This is the check I would defend hardest. CI regenerates the TypeScript client from the live schema and fails if the committed types differ. A response shape that changes without the frontend changing is a red build, not something a user discovers. It is structural: it does not depend on anybody remembering.

If he asks how you know the tests are worth anything, this is the best question you can get, have the answer ready:

By what they caught. Four examples.

Our macro-F1 was averaging over aspects with **zero support**, so a perfect system scored 0.22 on a two-topic set.

Our evaluation was lying to us in our own favour and a test caught it.

A connector was recording our own search term as though the source had said it, which means we would have been **citing ourselves as evidence**.

An unordered `LIMIT` meant ingest and fusion silently disagreed about which variant they meant.

And a migration would have failed against the 129 verdict rows already in production, because it emitted `NOT NULL` with no default.

4. Non-functional requirements (target 4 minutes, the longest section)

Show: `docs/non-functional-requirements.md`.

4.1 Open with the architecture, not the list

We audited all nine categories against the actual code rather than against what our proposal claimed, and wrote it up as this document.

The first thing to say is that six of the nine were already satisfied by a single architectural decision, and I would rather explain that decision than read you a list.

The write path and the read path are strictly separated. No model runs during a user request. Fusion, scoring, confidence intervals and claim generation all happen in the nightly pipeline and land as finished rows. Every endpoint is one indexed read of a row that already exists.

That one decision gives us four requirements at once. **Performance**, because nothing on the read path grows with the corpus, since the corpus is not on the read path. **Scalability**, because going from fifteen thousand review units to a hundred thousand changes how long the night takes and changes nothing about a request. **Reliability**, because a page cannot fail when a third-party model is down. And **availability**, because a demo depends on nothing remote.

Pause here. This is the sentence the mark rests on.

4.2 Then the five gaps

Reading the code against the nine categories found five real gaps, and we closed all five.

There were no security headers anywhere. Neither Vercel nor FastAPI adds any by default, so the site was shipping sniffable, framable, and sending a full referrer to every outbound link, and every outbound link we have points at a review on somebody else's site. Both now send a content security policy, nosniff, frame denial, referrer policy and HSTS.

No rate limiting, so one scraper walking every variant id could exhaust the connection pool every other reader was queued behind. It is now a sliding window, and it publishes the remaining budget so a well-behaved client can slow down before it is refused.

No compression or caching. Every client was re-fetching the full catalogue uncompressed. That is **55 kB down to under 6 kB**, about nine times smaller, which is what a phone on a weak connection waits on.

And no request identity, so a failure report could not be tied to a request. Every response now carries a request id and a server-side duration, which also makes our p95-under-300ms target checkable from outside instead of only by a benchmark we ran on ourselves.

4.3 The story to lead with

If you only get one NFR point across, make it this one.

The fourth gap is the one I would most like to tell you about, because it was not a missing feature. It was a requirement we had already implemented, and tested, and it still was not met.

Our health endpoint is deliberately written to report an unreachable database as a 503 rather than a 500 with a stack trace. It opens its own session specifically so the failure is a value it can report instead of an exception thrown before the handler even runs. It was written that way on purpose. There is a test for it.

It could not actually do it. There was no connect timeout, so the driver sat on the operating system's TCP timeout while the hosting platform's health probe gave up first. **A correct 503 became no answer at all**, and a hang is worse than an error, because nothing downstream can react to it.

What led us there was our own test run hanging for four hundred seconds against a machine with no Postgres. Every database wait is now bounded: connect at five seconds, pool wait at ten.

The lesson we took is the difference between writing a non-functional requirement and verifying one. We had the code, the intent, and the test, and the requirement was still not met.

4.4 Close with the limits

Do this before he finds them. It reads as judgement, not as a gap.

Two limits we state in the document rather than hide.

The rate limiter is per instance and in memory, so it is not a global quota. Two instances would each allow the full rate. That is the right shape for the risk we actually have, which is one scraper exhausting a connection pool, not a distributed attack. A global quota needs Redis, and adding a second network dependency to the read path in order to protect the read path is a bad trade at this size.

And a single free-tier instance is resilient to its dependencies failing, not highly available. There is no redundancy and no failover. Buying that means at least two instances and a load balancer.

Both are the first things that would need replacing if this grew.

5. Closing (20 seconds)

So: about fifteen thousand lines running on real data, with the framework choices written down as decision records including the ones we rejected, quality enforced by six CI jobs rather than by intention, and all nine non-functional categories audited with five gaps found and closed.

The working demo is the next submission, and the site is live now if you want to look at it before then.

Likely questions, and honest answers

"Why is the classifier disabled? Isn't the ML the point?"

The point is that a number should be earned. We measured it against the lexicon it was trained from and it lost by 0.29 macro F1. Shipping it anyway would contradict the whole argument of the project. The code, the result and the reason are all kept, and it is published on our metrics endpoint rather than quietly dropped.

"Is 246 tests not a lot for a student project? Are they real?"

Judge them by what they caught rather than the count. An evaluation bug that was flattering us, a connector citing our own search term as evidence, an unordered LIMIT that made two stages disagree, and a migration that would have failed against production rows.

"Why not Airflow?"

One nightly batch with no fan-out and no backfill. Actions already holds our secrets, already schedules, already alerts. ADR 0002 records the decision and the trigger for revisiting it.

"Your p95 target is 300ms. Have you actually measured it?"

Yes, and we made it measurable from outside. Every response carries an `X-Response-Time-ms` header. Warm and local it is 22 to 35 milliseconds; from India against the deployed instance it is 96 to 247, inside the target including network. The reason it holds is architectural, not tuning: there is nothing on the read path to be slow.

"What about the cold start on a free tier?"

That is real and we do not hide it. A scheduled workflow pings the instance to keep it out of that state, and our cache directives let a client render a recent copy while the API wakes. It is a platform property, not a code path, and it is written into the availability section.

"Only three sources?"

Three that permit us. Team-BHP returns 403 to our agent, and two other sites disallow us outright in robots.txt. **We chose to respect that rather than spoof a browser**, and the connectors identify honestly and rate-limit themselves. Working around it would need us to misrepresent who we are, and that is a third party's decision to make, not ours to route around.

"Is any of this data fabricated?"

None of it. 15,853 real review units. Where a number does not exist we say so: nine variants sit below our evidence floor and are held back rather than shown, and the site says why.

If something goes wrong

Problem	What to do
The live site is cold	Keep talking; it wakes in a few seconds. Or say "this is the free-tier cold start I mentioned" and turn it into the availability point
CI is red on the day	Open the failing job and read it out. A red build you can diagnose in ten seconds is a better answer than a green one you cannot explain
He asks for something not built	"Not built, and here is what we chose instead and why." Never invent a feature
You lose the thread	Go back to the four marked headings: codebase, frameworks, quality, NFRs